

Optimization

6.3 Aggregation (scalarization) methods

José Firmino Aguilar Madeira

The simplest idea: back to one objective

We have $\min_{\mathbf{x} \in X} (f_1(\mathbf{x}), f_2(\mathbf{x}))$. Why not choose the importance of each objective and solve

$$\min_{\mathbf{x} \in X} w_1 f_1(\mathbf{x}) + w_2 f_2(\mathbf{x})$$

with $w_1, w_2 \geq 0$, $w_1 + w_2 = 1$?

The simplest idea: back to one objective

We have $\min_{\mathbf{x} \in X} (f_1(\mathbf{x}), f_2(\mathbf{x}))$. Why not choose the importance of each objective and solve

$$\min_{\mathbf{x} \in X} w_1 f_1(\mathbf{x}) + w_2 f_2(\mathbf{x})$$

with $w_1, w_2 \geq 0$, $w_1 + w_2 = 1$?

We are back to a problem with a single objective:
we can use everything we learned in Chs. 2–5.

The simplest idea: back to one objective

We have $\min_{\mathbf{x} \in X} (f_1(\mathbf{x}), f_2(\mathbf{x}))$. Why not choose the importance of each objective and solve

$$\min_{\mathbf{x} \in X} w_1 f_1(\mathbf{x}) + w_2 f_2(\mathbf{x})$$

with $w_1, w_2 \geq 0$, $w_1 + w_2 = 1$?

We are back to a problem with a single objective: we can use everything we learned in Chs. 2–5.

We already did it in 6.1

Nantes–Lille, “an hour is worth 30 €”:

$$\text{cost} + 30 \times \text{duration}$$

is a **weighted sum** of the two criteria.

Gass and Saaty (1955); with m objectives,
 $\min \sum_i w_i f_i, \sum_i w_i = 1$.

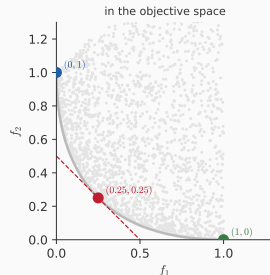
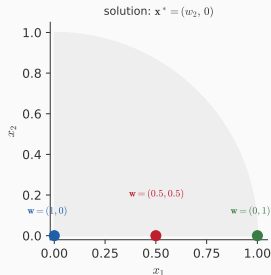
What do the weights do?

In the example of the chapter:

$$\min w_1(x_1^2 + x_2^2) + w_2[(x_1 - 1)^2 + x_2^2].$$

Differentiating: $x_2^* = 0$, $x_1^* = w_2$.

$\mathbf{w}$	$\mathbf{x}^*$
$(1, 0)$	$(0, 0)$
$(0.5, 0.5)$	$(0.5, 0)$
$(0, 1)$	$(1, 0)$



What do the weights do?

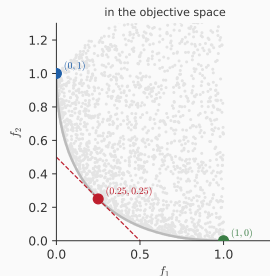
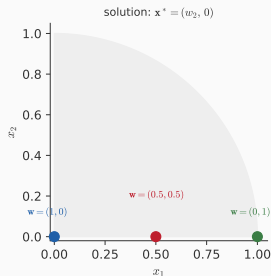
In the example of the chapter:

$$\min w_1(x_1^2 + x_2^2) + w_2[(x_1 - 1)^2 + x_2^2].$$

Differentiating: $x_2^* = 0$, $x_1^* = w_2$.

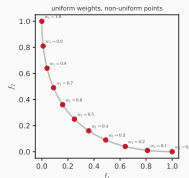
$\mathbf{w}$	$\mathbf{x}^*$
$(1, 0)$	$(0, 0)$
$(0.5, 0.5)$	$(0.5, 0)$
$(0, 1)$	$(1, 0)$

If we vary the weights, do we sweep the front? In this example, yes: $x_1^* = w_2$ sweeps the whole segment X_E .



First surprise: uniform weights $\neq$ uniform points

w_1	w_2	x_1^*	f_1	f_2
0.0	1.0	1.000	1.000	0.000
0.1	0.9	0.900	0.810	0.010
0.2	0.8	0.800	0.640	0.040
0.3	0.7	0.700	0.490	0.090
0.4	0.6	0.600	0.360	0.160
0.5	0.5	0.500	0.250	0.250
0.6	0.4	0.400	0.160	0.360
0.7	0.3	0.300	0.090	0.490
0.8	0.2	0.200	0.040	0.640
0.9	0.1	0.100	0.010	0.810
1.0	0.0	0.000	0.000	1.000



In the decisions $x_1^* = w_2$ is uniform; in the objectives, $f_1 = w_2^2$, $f_2 = (1 - w_2)^2$: the points **are not equally spaced** (closer together in the middle).

The weights control the scalarization; **they do not directly control the position on the front.**

With $m \geq 3$, even less predictable. Cost here: 11 points, $n_f = 75$ (SLSQP, the solver's forward-difference gradients, warm start).

Second surprise: scale matters

Three designs (illustrative numbers): $f_1 = \text{mass}$ (kg), $f_2 = \text{cost}$ (€).

	mass	cost
A	1.5	800
B	0.9	850
C	0.5	1000

With $\mathbf{w} = (0.9, 0.1)$ — “90% mass” — without normalizing: $A = 81.4$, $B = 85.8$, $C = 100.5$: **A wins, the heaviest**. The cost, in the hundreds, dominates the sum.

Second surprise: scale matters

Three designs (illustrative numbers): $f_1 = \text{mass}$ (kg), $f_2 = \text{cost}$ (€).

	mass	cost
A	1.5	800
B	0.9	850
C	0.5	1000

With $\mathbf{w} = (0.9, 0.1)$ — “90% mass” — without normalizing: $A = 81.4$, $B = 85.8$, $C = 100.5$: **A wins, the heaviest**. The cost, in the hundreds, dominates the sum.

Normalize with ideal & nadir (6.2)

$$\hat{f}_i(\mathbf{x}) = \frac{f_i(\mathbf{x}) - z_i^{id}}{\hat{z}_i^{nad} - z_i^{id}} \in [0, 1] \text{ on the efficient set;}$$
$$\min \sum_i w_i \hat{f}_i(\mathbf{x}).$$

Here $\mathbf{z}^{id} = (0.5, 800)$, $\hat{\mathbf{z}}^{nad} = (1.5, 1000)$:

$\mathbf{w}$	(0.9, 0.1)	(0.5, 0.5)	(0.1, 0.9)
chooses	C	B	A

Before interpreting the weights as relative importance, **normalize** objectives with different scales.

What can we guarantee?

Property

If $w_i > 0$ for all i and $\mathbf{x}^*$ minimizes $\sum_i w_i f_i$ on X , then $\mathbf{x}^*$ is **efficient**.

What can we guarantee?

Property

If $w_i > 0$ for all i and $\mathbf{x}^*$ minimizes $\sum_i w_i f_i$ on X , then $\mathbf{x}^*$ is **efficient**.

One-line proof. If there were $\mathbf{x} \in X$ dominating $\mathbf{x}^*$ — $f_i(\mathbf{x}) \leq f_i(\mathbf{x}^*)$ for all i and $<$ for some —, then, since all the weights are positive,

$$\sum_i w_i f_i(\mathbf{x}) < \sum_i w_i f_i(\mathbf{x}^*),$$

contradicting the minimality of $\mathbf{x}^*$. □

What can we guarantee?

Property

If $w_i > 0$ for all i and $\mathbf{x}^*$ minimizes $\sum_i w_i f_i$ on X , then $\mathbf{x}^*$ is **efficient**.

One-line proof. If there were $\mathbf{x} \in X$ dominating $\mathbf{x}^*$ — $f_i(\mathbf{x}) \leq f_i(\mathbf{x}^*)$ for all i and $<$ for some —, then, since all the weights are positive,

$$\sum_i w_i f_i(\mathbf{x}) < \sum_i w_i f_i(\mathbf{x}^*),$$

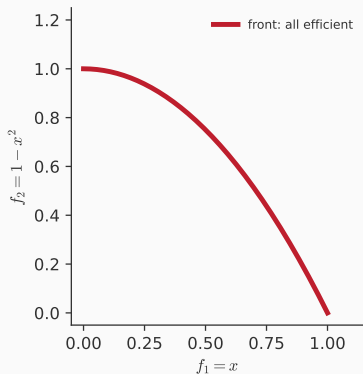
contradicting the minimality of $\mathbf{x}^*$. □

With some $w_i = 0$ that objective no longer counts: in general only *weak* efficiency is guaranteed. Hence, in what follows, $w_i > 0$ (as in the definition of *supported*, 6.2).

The fundamental limitation: a concave front

$x \in [0, 1]$, $f_1(x) = x$, $f_2(x) = 1 - x^2$ (the nonconvex example from 6.2, with $x_2 = 0$). Front: $f_2 = 1 - f_1^2$, concave.

All these points are efficient. If we vary w_1 from 0 to 1, can we find them all?



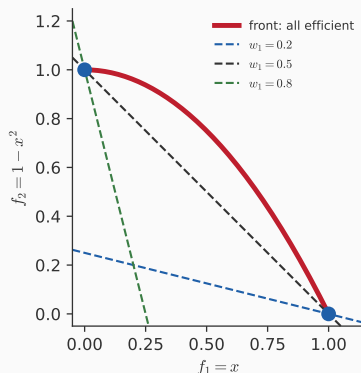
The fundamental limitation: a concave front

$x \in [0, 1]$, $f_1(x) = x$, $f_2(x) = 1 - x^2$ (the nonconvex example from 6.2, with $x_2 = 0$). Front: $f_2 = 1 - f_1^2$, concave.

We try $w_1 = 0.2, 0.5, 0.8$: the level lines always select **an extreme**.

11 weights $\rightarrow$ 2 points. 100 weights $\rightarrow$ 2 points.

10 000 weights $\rightarrow$ 2 points.



The fundamental limitation: a concave front

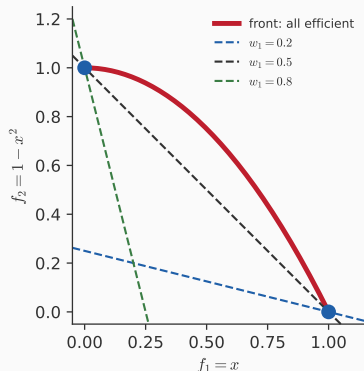
$x \in [0, 1]$, $f_1(x) = x$, $f_2(x) = 1 - x^2$ (the nonconvex example from 6.2, with $x_2 = 0$). Front: $f_2 = 1 - f_1^2$, concave.

Why? With $w_1, w_2 > 0$,

$$\tilde{f}(x) = w_1 x + w_2(1 - x^2), \quad \boxed{\tilde{f}''(x) = -2w_2 < 0}.$$

$\tilde{f}$ is **concave**: on an interval, the minimum is at one of the endpoints, $x^* \in \{0, 1\}$, that is, $(0, 1)$ or $(1, 0)$.

All the points with $0 < x < 1$ are efficient, but *non-supported*.



The fundamental limitation: a concave front

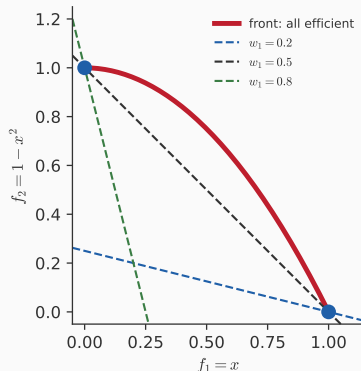
$x \in [0, 1]$, $f_1(x) = x$, $f_2(x) = 1 - x^2$ (the nonconvex example from 6.2, with $x_2 = 0$). Front: $f_2 = 1 - f_1^2$, concave.

Why? With $w_1, w_2 > 0$,

$$\tilde{f}(x) = w_1 x + w_2(1 - x^2), \quad \boxed{\tilde{f}''(x) = -2w_2 < 0}.$$

$\tilde{f}$ is **concave**: on an interval, the minimum is at one of the endpoints, $x^* \in \{0, 1\}$, that is, $(0, 1)$ or $(1, 0)$.

All the points with $0 < x < 1$ are efficient, but *non-supported*.



It is not a matter of choosing too few weights: **no positive weight finds the interior points.**

Weighted sum: the full picture

$$\min_{\mathbf{x} \in X} \sum_{i=1}^m w_i f_i(\mathbf{x}), \quad w_i > 0, \quad \sum_i w_i = 1.$$

Always

the **global** minimizer is **efficient**.

Convex problem

every efficient point minimizes the sum for some $\mathbf{w} \geq \mathbf{0}$, $\mathbf{w} \neq \mathbf{0}$; the properly efficient ones (bounded trade-off rates), for some $\mathbf{w} > \mathbf{0}$ (Geoffrion, 1968) (limit points such as the extremes of the convex example may require a zero weight).

Nonconvex problem *only the supported points.*

We have already seen all three cases happen on the previous pages.

Another aggregation: weighted Tchebycheff

The **weighted distance to the ideal** in the max norm:

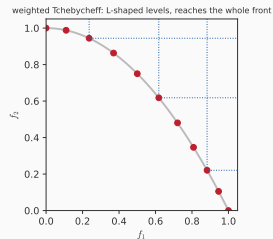
$$\min_{\mathbf{x} \in X} \max_i w_i (f_i(\mathbf{x}) - z_i^{id}).$$

Geometrically: weighted sum $\rightarrow$ **straight lines**;

Tchebycheff $\rightarrow$ **L-shaped corners** whose vertices lie on the ray from the ideal in the direction $(1/w_1, 1/w_2)$, which can also touch the *non-supported* regions.

The *non-supported* limitation belongs to the weighted sum, not to all scalarizations.

	weighted sum	Tchebycheff
scalar problem	sum	maximum
<i>non-supported</i> points	no	yes
position on the front	hard to control	hard to control
differentiable	if the f_i are	not directly



Notes

Augmented version, $+\rho \sum_i (f_i - z_i^{id})$ with small $\rho > 0$: avoids solutions that are only weakly efficient. Without the max (4.2): $\min \theta$ s.t. $w_i (f_i - z_i^{id}) \leq \theta$. Other aggregations: goal programming, lexicographic.

The price of aggregation

Advantages

- very simple;
- reuses any method of Chs. 2–5 and its theory;
- incorporates the decision-maker's preferences;
- one optimization gives one solution.

Limitations

- one optimization **per point**;
- the weights depend on the scale;
- uniform weights do not give uniform coverage;
- position on the front hard to control;
- the weighted sum misses the *non-supported* points.

To build a front, we have to **repeat the optimization** — once per point.

KEY TAKEAWAYS — 6.3

1. Aggregation turns several objectives into a scalar problem: $\min \sum_i w_i f_i$.
2. With $w_i > 0$, a **global** minimizer of the weighted sum is **efficient**.
3. Objectives with different scales must be **normalized** before the weights are interpreted.
4. Uniform weights $\neq$ uniform points on the front; in nonconvex problems, the weighted sum **does not find** *non-supported* points.
5. Other scalarizations (Tchebycheff) reach the *non-supported* points, but still give **one point per optimization**.

What if, instead of combining the objectives in a sum, we keep one as the objective and turn the others into constraints?



6.4 ϵ -constraint

References Full list: *Course bibliography*, at the end of the course PDF.

- Miettinen, K., *Nonlinear Multiobjective Optimization*, Kluwer Academic Publishers, 1999.
- Geoffrion, A. M., *Proper efficiency and the theory of vector maximization*, Journal of Mathematical Analysis and Applications 22(3):618–630, 1968.
- Jahn, J., *Vector Optimization: Theory, Applications, and Extensions*, 2nd ed., Springer, 2011 — Ch. 5.

Resources: costs computed with SLSQP (SciPy); weighted-sum code (`fmincon`) in 6.4.